

YourTodo AI 기반 개발 사례 소개서

1. 한 줄 소개

YourTodo는 AI를 단순 코드 자동완성 도구로 쓰지 않고, 제품 기획, 기술 설계, 모듈 구현, 테스트, 문서화, 품질 검토 까지 이어지는 agentic development 방식으로 만든 Android 앱이다.

이 프로젝트에서 AI는 “코드를 대신 치는 도구”보다 “개발 시스템 안에서 반복 가능한 작업을 수행하는 협업 에이전트”에 가깝다. 기능 요구사항을 PRD/TRD로 정리하고, Android 멀티모듈 구조 안에서 구현 범위를 나누고, hook, CI, coverage, Gradle 테스트와 lint 같은 하네스로 결과를 확인하며, 마지막에는 외부 공유 문서까지 생성하는 흐름을 갖췄다.

2. 최신 AI 코딩 트렌드와 맞닿은 지점

AI 코딩 도구의 흐름은 빠르게 agentic coding으로 이동하고 있다. 최신 흐름을 요약하면 다음과 같다.

트렌드	의미	YourTodo에서의 연결
Agentic coding	AI가 파일을 읽고, 수정하고, 명령을 실행하며 작업을 끝까지 수행	모듈 분석, 코드 변경, 테스트 실행, 문서화까지 한 흐름으로 처리
Multi-file / multi-module editing	단일 파일 자동완성을 넘어 여러 파일과 모듈을 함께 변경	feature/api/entry/impl, core/domain/data/network/database를 함께 설계
Test harness 중심 개발	AI 결과를 테스트, lint, build로 검증	Gradle module test, lint, Room/DTO/ViewModel test 기반
Repository-aware agents	코드베이스 전체 구조와 팀 규칙을 읽고 작업	AGENTS.md, 모듈 가이드, PRD/TRD, 기존 패턴 기반 구현
Skills / instructions / hooks	반복 작업을 팀 규칙과 도구로 패키징	AGENTS.md, .husky, scripts/codex-hooks, review checklist 활용
Parallel / delegated agents	여러 에이전트가 독립 작업을 병렬 수행	분석, 구현, 검증, 문서화를 분리 가능한 단위로 설계
Human-in-the-loop review	AI 결과를 사람이 승인하고 제품 판단을 유지	AI Todo Draft도 사용자 review sheet를 거쳐 저장

OpenAI Codex는 cloud sandbox에서 코드 읽기/수정/테스트 실행과 병렬 작업을 강조하고, GitHub Copilot agent mode는 여러 파일 변경과 조직별 지시사항을 강조한다. Claude Code 문서는 코드베이스를 읽고 명령을 실행하며 MCP, instructions, hooks, subagents를 연결하는 흐름을 소개한다. Google DORA의 AI-assisted software development 보고서는 AI 도입을 개별 도구가 아니라 개발 시스템과 팀 역량의 문제로 다룬다.

최근 OpenAI와 Anthropic의 하네스 관련 글에서 공통으로 드러나는 방향은 “AI에게 더 큰 자유를 주려면, 그만큼 더 명확한 환경과 검증 루프를 제공해야 한다”는 점이다.

최신 흐름	핵심 메시지	YourTodo에 반영된 방식
OpenAI Harness Engineering	agent-first 개발에서는 코드보다 scaffolding, feedback loop, control system의 중요성이 커짐	AGENTS.md, PRD/TRD, hook, CI, coverage gate를 저장소의 작업 레일로 둠
OpenAI SWE-bench Verified	coding agent 평가는 failing test와 regression test가 함께 있어야 실제 수정을 판별 가능	버그 수정 시 회귀 테스트, 영향 모듈 테스트, coverageVerifyAI로 기능/회귀를 함께 확인
OpenAI SWE-bench 평가 회고	benchmark도 오염되거나 테스트가 좁고 넓어질 수 있어 평가 자체를 계속 점검해야 함	PR template과 리뷰 기준으로 테스트가 요구사항과 맞는지 함께 기록
Anthropic Claude Code Best Practices	instructions는 advisory지만 hooks는 deterministic하게 반드시 실행되는 장치	.husky, scripts/git-hooks, scripts/codex-hooks/tdd-guard.sh로 반복 규칙을 실행 가능하게 만들
Anthropic Agent Evals	agent 품질은 capability eval과 regression eval을 분리해 지속적으로 측정해야 함	새 기능 검증과 기존 기능 회귀 방지를 Gradle test/lint/CI에서 함께 수행
Anthropic Long-running Agent Harness	긴 작업은 초기화, 중분 진행, 다음 세션을 위한 명확한 산출물이 중요	PRD/TRD, follow-up 문서, worktree, 변경 요약으로 컨텍스트를 이어받을 수 있게 함

3. YourTodo에서 AI가 맡은 역할

3.1 제품 기획 파트너

AI는 기능 아이디어를 바로 코드로 옮기기보다 먼저 제품 요구사항을 정리하는 데 사용됐다.

산출물	역할
PRD	사용자 문제, 목표, 비목표, 수용 기준 정리
TRD	모듈 영향, 데이터 계약, API, 테스트 계획 정리
디자인/플로우 문서	화면 책임, 상태, 사용자 경로 정리
아키텍처 리포트	현재 구조, 데이터 흐름, 품질 기준 정리

이 과정은 AI가 “무엇을 만들지”를 명확히 한 뒤 “어디를 고칠지”를 찾도록 만드는 역할을 했다.

3.2 코드베이스 탐색자

AI는 구현 전에 모듈 구조, 의존 방향, 기존 패턴을 먼저 읽었다.

```
요구사항 확인
-> root AGENTS.md와 모듈 AGENTS.md 확인
-> settings.gradle.kts로 모듈 구조 파악
-> 기존 use case / repository / ViewModel 패턴 확인
-> 변경 범위 결정
```

이 덕분에 기능 구현이 기존 아키텍처에 맞게 들어갔다. 예를 들어 새로운 기능은 feature 구현체가 data 구현체를 직접 참조하지 않고, domain use case와 repository contract를 통해 연결되는 형태를 유지한다.

3.3 설계-구현-검증 루프

AI 활용 흐름은 다음처럼 반복됐다.

```
Plan
-> PRD/TRD 또는 구현 계획 작성
Explore
-> 관련 모듈과 기존 테스트 확인
Implement
-> 코드 변경
Verify
-> 단위 테스트, lint, build, PDF/문서 렌더 확인
Review
-> 변경 요약, 남은 판단, 외부 공유 문서화
```

이 루프는 최신 AI 코딩 도구들이 강조하는 “에이전트가 명령을 실행하고 하네스로 결과를 검증한다”는 방향과 맞닿아 있다.

4. 프로젝트가 갖춘 AI-friendly 구조

YourTodo는 AI가 안정적으로 작업하기 좋은 구조를 갖고 있다.

구조	AI 개발에서의 장점
멀티모듈 Gradle	변경 범위를 모듈 단위로 좁히고 검증 가능
Feature API/Entry/Impl	route 계약과 구현을 분리해 영향 범위 명확
Domain use case	AI가 비즈니스/앱 정책을 화면 밖에서 다루기 쉬움
Repository contract	fake/test double 작성이 쉬움
Room/DataStore/Retrofit 분리	저장소별 책임이 명확해 분석과 테스트가 쉬움
UiState/Action/SideEffect	화면 상태 변경이 예측 가능
PRD/TRD 문서	AI가 요구사항을 재사용 가능한 컨텍스트로 읽을 수 있음
AGENTS.md	팀 규칙, 모듈 경계, 검증 명령을 AI에게 전달

AI는 구조가 흐릿한 코드베이스보다 경계가 명확한 코드베이스에서 더 강해진다. YourTodo는 AI가 읽고, 수정하고, 검증할 단위가 비교적 선명하다.

5. 사용한 스킬과 하네스

5.1 스킬

이 프로젝트에서 중요한 AI 활용 스킬은 다음과 같다.

스킬	설명
Architecture reading	settings.gradle.kts, AGENTS.md, module guide, 기존 구현을 읽고 작업 경계 파악
Specification writing	PRD/TRD로 기능 목표, 비목표, API, 테스트 전략 정리
Android implementation	Kotlin, Coroutines, Compose, Hilt, Navigation, Room, DataStore, Retrofit 적용
Data modeling	DTO, Entity, Domain model, mapper, migration, sync payload 설계
UDF state modeling	UiState + Action + SideEffect로 화면 상태 설계
Test-first thinking	use case/ViewModel/repository/DAO 단위 테스트 우선 검토
Regression review	변경으로 깨질 수 있는 사용자 흐름과 데이터 계약 점검
Documentation generation	내부 진단 문서와 외부 공유 문서, PDF 산출

5.2 하네스

하네스는 AI가 만든 결과를 검증하는 장치다. YourTodo는 다음 하네스를 갖고 있다.

하네스	검증 대상
Gradle module unit test	domain/data/feature 단위 동작
Android lint	모듈별 Android 품질 규칙
Room DAO/migration test	DB query, schema migration, FK/캐시 정책
ViewModel test	UiState, action, side effect
Network DTO serialization test	서버 계약과 JSON payload
DataStore policy test	auth token, preference migration
Widget presenter/content test	Glance state와 표시 모델
PDF render check	문서 산출물의 한글/표/코드블록 렌더링

이 하네스가 있기 때문에 AI는 단순히 변경을 제안하는 데서 멈추지 않고, 변경 결과를 확인하는 쪽으로 이어질 수 있다.

5.3 저장소에 포함된 제품 하네스

YourTodo에는 AI와 사람이 같은 품질 기준으로 작업하도록 돕는 자동화 장치가 저장소 안에 포함되어 있다.

하네스	실제 구성	품질 효과
.husky/pre-commit	scripts/git-hooks/pre-commit.sh 실행	main/master 직접 commit을 막고 최신 origin/main 위에서 작업하게 함
.husky/pre-push	scripts/git-hooks/pre-push.sh 실행	main/master 직접 push를 막고, 최신 main 포함 여부와 변경 영향 lint를 확인
제품 하네스 검사	scripts/quality/product-harness-check.sh	모듈별 AGENTS.md, 루트 모듈 인덱스, Gradle 의존 방향, 금지 import를 자동 검증
재작업 관측성 검사	scripts/quality/rework-metrics-check.sh	PR review thread, PR body, follow-up commit, branch metrics 문서가 서로 맞지 않으면 실패

하네스	실제 구성	품질 효과
영향 범위 lint	변경 파일을 app/core/feature 모듈로 분류	작은 변경은 관련 모듈 lintDebug, Gradle/CI/hook 변경은 전체 lint로 검증
Android CI	.github/workflows/android-ci.yml	testDebugUnitTest, assembleDebug, 전체 lint, coverageVerifyAll을 PR/push에서 실행
CI artifact	test/lint/build/ Jacoco report 업로드	실패 원인과 품질 지표를 PR에서 추적 가능하게 함
Strict lint	root Gradle에서 abortOnError, warningsAsErrors, checkAllWarnings, checkDependencies 설정	Android warning을 조기 실패로 바꿔 품질 편차를 줄임
Coverage gate	coverageReportAll, coverageVerifyAll	핵심 non-view 모듈의 line coverage 기준을 자동 검증
TDD Guard	scripts/codex-hooks/tdd-guard.sh	운영 코드 변경 전 같은 모듈 테스트 변경을 요구
Worktree bootstrap	scripts/start-worktree-from-main.sh	최신 main 기준 새 브랜치와 별도 worktree를 생성
PR template	.github/pull_request_template.md	문제, 범위, 설계, 테스트, 커버리지, 리스크, 마이그레이션을 매 PR에 남김

이 구성은 AI가 빠르게 수정하더라도 변경이 main에 섞이기 전에 여러 지점에서 걸러지게 만든다. local hook은 개발자 작업 순간을 막고, 제품 하네스 검사는 구조 경계를 빠르게 확인하며, CI는 원격 PR 기준으로 다시 검증한다. coverage와 PR template은 테스트 결과와 리스크 설명을 남기게 한다.

5.4 AGENTS.md 제약의 품질 효과

AGENTS.md는 단순 설명서가 아니라 AI 에이전트와 개발자가 따라야 할 작업 제약이다. 이 제약은 제품 품질을 높이는 방향으로 설계되어 있다.

제약	품질 효과
전용 브랜치와 별도 worktree에서 작업	기본 체크아웃 오염과 main 직접 변경을 줄임
최신 main 기준 작업	오래된 기준에서 생기는 충돌과 회귀를 줄임
영향 모듈 단위 테스트와 lint 필수	변경 범위에 맞는 빠른 검증을 습관화
버그 수정 시 회귀 테스트 필수	같은 실패가 다시 들어오는 것을 막음
non-view 레이어 80% 이상 커버리지 목표	domain/data/ViewModel 로직을 테스트 가능한 형태로 유지
core:* -> feature:* 의존 금지	공통 계층이 기능 구현에 끌려가지 않게 함
feature:*.api와 feature:*.impl 분리	공개 계약과 구현 세부사항을 분리해 변경 안정성 확보
UiState + Action + SideEffect 우선	화면 상태와 1회성 이벤트를 예측 가능하게 관리
사용자 노출 문자열 리소스화	로케일 대응과 UI 테스트 안정성 확보
Room/DataStore/Network 계약 변경 시 호환성 검증	데이터 손실과 서버 계약 회귀를 줄임
PR 리뷰 P1 기준 명시	아키텍처 위반, 테스트 누락, 리소스 하드코딩을 일관되게 차단

이 제약의 핵심은 AI에게 “많이 바꾸라”가 아니라 “작게 바꾸고, 올바른 경계에서 바꾸고, 검증 가능한 방식으로 바꾸라”는 작업 레일을 제공한다는 점이다.

5.5 OpenAI/Anthropic 하네스 관점의 프로젝트 인사이트

OpenAI의 Harness Engineering 글은 agent-first 개발에서 중요한 일이 “좋은 코드를 직접 쓰는 것”에서 “좋은 코드가 나오도록 환경, 추상화, 피드백 루프를 설계하는 것”으로 이동하고 있음을 보여준다. Anthropic의 Claude Code와 Agent Evals 글도 같은 방향을 가리킨다. AI가 자율적으로 움직일수록 테스트, hook, 권한, 회귀 평가, 명시적 작업 기록이 더 중요해진다.

YourTodo의 개발 방식은 이 흐름과 잘 맞는다.

인사이트	YourTodo에서의 구체화
Repository is the harness	저장소 안에 AGENTS.md, 모듈별 가이드, PRD/TRD, playbook, hook, CI가 함께 있어 AI가 같은 규칙을 반복 적용
Instructions need executable guards	규칙을 문서에만 두지 않고 .husky, pre-push lint, TDD Guard, coverage gate로 실행 가능하게 만들
Regression must be explicit	버그 수정은 회귀 테스트를 요구하고, CI는 기존 테스트와 lint를 다시 실행해 이전 동작을 보호
Eval quality matters	PR template과 review guideline이 테스트/커버리지/마이그레이션 근거를 요구해 “통과는 했지만 의미가 약한 테스트”를 줄임
Context continuity matters	PRD/TRD와 follow-up 문서가 장기 작업의 상태를 남겨 다음 AI 세션이나 사람이 같은 맥락에서 이어갈 수 있게 함
Rework must be observable	review, CI, follow-up commit이 branch metrics 문서와 PR 요약에 남아 재작업 원인과 자동화 가능성을 추적
Human judgment moves up a layer	사람은 코드 줄 단위보다 제품 의미, 우선순위, 예외 정책, 외부 공유 메시지를 판단하고 AI는 실행과 검증 루프를 담당

이 관점에서 YourTodo의 hook, CI, 품질 게이트는 단순 개발 편의가 아니라 제품 품질 인프라다. AI가 만드는 변경을 빠르게 받아들이되, 모듈 경계, 테스트 선행, lint, coverage, PR 설명, 리뷰 기준을 통과한 변경만 제품에 가까워지도록 만드는 구조이기 때문이다.

6. 대표 AI 협업 사례

6.1 Todo sync

AI는 local-first Todo 구조를 서버 동기화 구조로 확장하는 데 사용됐다.

```
Room todo
+ todo_outbox
+ sync_cursor
+ pull/push/pull coordinator
+ DTO/domain/entity mapper
```

이 기능은 단순 CRUD가 아니라 오프라인 변경, 서버 수렴, auth refresh, mutation 결과 처리를 함께 다룬다. AI는 TRD로 정책을 먼저 정리한 뒤 repository, DAO, DataStore, network DTO, 테스트를 연결하는 방식으로 구현 흐름을 도왔다.

6.2 Task Surface

Todo, Calendar, Widget이 같은 할 일 현실을 보도록 TaskSurface domain 정책을 만들었다.

```
local Todo + assigned Todo
-> TaskSurfaceItem
-> list / date agenda / monthly summary
-> Todo, Calendar, Widget UI
```

AI는 중복된 화면별 합성 로직을 domain use case로 모으고, filter/sort/date/assigned override 테스트 matrix를 확장하는 데 기여했다.

6.3 Direct Assignment

Direct Assignment는 제품 정책, API 계약, Android UI, Room cache, notification routing이 함께 움직이는 기능이다. AI는 PRD/TRD로 권한 방향과 사용자 노출 용어를 정리하고, REQUEST와 DIRECT가 Todo/Calendar/Widget/Friends에서 일관되게 보이도록 흐름을 설계했다.

6.4 Person Todo Visibility

Person Todo Visibility는 AI 협업에서 제품 의미 분리가 얼마나 중요한지 보여주는 사례다. 기능은 “친구에게 할 일을 주는 것”이 아니라 “친구가 허용한 할 일 흐름을 읽기 전용으로 보는 것”이므로, PRD/TRD 단계에서 Shared Todo, Direct Assignment, ObservedTodo, VisibilityGrant를 명확히 분리했다.

AI는 이 분리를 코드 구조에도 반영했다. `core:model`에는 `ObservedTodo`와 `PersonVisibilityGrant`를 두고, `core:domain`은 `PersonVisibilityRepository`와 use case를 제공하며, `core:data`는 Room cache와 Retrofit API를 연결한다. `Friends`는 활성 친구 row 안에서 친구 할일을 펼치고, `Calendar`는 observed item이 있는 날짜에만 읽기 전용 섹션을 더한다. `Widget`과 `Todo tab`은 의도적으로 제외해 "내 할일"의 의미를 보존한다.

6.5 AI Todo Draft

앱 자체에도 AI 기능이 들어 있다. 사용자는 한국어 자연어 문장으로 여러 할 일을 입력하고, 앱은 AI proxy를 통해 구조화된 초안을 만든다.

중요한 점은 AI 결과를 곧바로 저장하지 않는다는 것이다. 앱은 review sheet를 제공하고, 사용자가 제목/담당자/날짜/시간/우선순위를 확인한 뒤 저장하게 한다. 이는 AI 기능을 제품 안에 넣을 때 신뢰와 통제권을 함께 설계한 사례다.

7. AI 개발 방식의 특징점

특장점	설명
명세 중심 개발	AI가 바로 코드로 가지 않고 PRD/TRD로 목표와 경계를 먼저 정리
모듈 경계 준수	<code>core:* -> feature:*</code> 역방향 의존을 피하고 기존 의존 방향 유지
테스트 가능한 설계	use case, repository, DAO, ViewModel 단위로 검증 가능
데이터 계약 가시화	sync payload, network DTO, Room entity, DataStore key를 문서화
Hook/CI 품질 레일	.husky, GitHub Actions, strict lint, coverage gate로 변경을 단계별 검증
AGENTS 정책 내재화	브랜치, 모듈 경계, TDD, 리뷰 기준을 AI 작업 제약으로 적용
Harness engineering 지향	문서, hook, CI, coverage, review 기준을 저장소 안에 넣어 AI가 반복 가능한 방식으로 작업
Agent eval 사고방식	기능 통과 테스트와 회귀 방지 테스트를 함께 보며 평가 품질까지 관리
Rework observability	리뷰/CI/후속 커밋을 branch metrics 문서로 reconcile해 개선 여부를 장기 측정
사용자 검토 유지	AI 초안 저장도 review-first UX로 설계
산출물 자동화	코드뿐 아니라 내부/외부 문서와 PDF까지 생성
변화 추적	main 변경을 다시 분석해 문서를 최신화하는 루프 보유

8. 개발 생산성 측면의 의미

YourTodo의 AI 활용 방식은 "한 번에 많은 코드를 생성"하는 방식보다 "반복 가능한 개발 절차를 빠르게 돌리는 방식"에 가깝다.

기획 문장
-> PRD
-> TRD
-> 모듈 영향 분석
-> 구현
-> hook/TDD guard
-> 테스트/린트/coverage/CI
-> 문서화
-> 외부 공유 자료

이 흐름은 기능이 늘어날수록 가치가 커진다. 새로운 기능을 추가할 때마다 같은 절차로 요구사항, 모듈 경계, 데이터 계약, 테스트 범위를 빠르게 정렬할 수 있기 때문이다.

9. AI와 사람의 역할 분담

역할	AI가 잘한 일	사람이 유지한 판단
제품 기획	문제/목표/수용 기준 구조화	최종 UX 방향과 우선순위

역할	AI가 잘한 일	사람이 유지한 판단
아키텍처	모듈 경계 분석, 영향 범위 추적	어떤 복잡도를 받아들일지 결정
구현	반복 코드, mapper, use case, 테스트 작성	제품 의미와 예외 정책 승인
검증	테스트 명령 실행, 실패 원인 추적	릴리즈 기준과 리스크 판단
문서화	리포트, 소개서, PDF 생성	외부에 어떤 메시지를 낼지 선택

AI는 개발자의 판단을 대체하기보다, 판단을 내릴 수 있는 자료를 빠르게 정리하고 실행하는 역할에 강하다.

10. 참고한 최신 AI 코딩 흐름

- OpenAI Codex 소개 (<https://openai.com/index/introducing-codex/>): cloud-based software engineering agent, 병렬 작업, 코드 읽기/수정/테스트 실행 흐름.
- OpenAI Codex 제품 페이지 (<https://openai.com/codex/>): multi-agent workflows, worktrees, Skills 기반 팀 워크플로우.
- GitHub Copilot agent mode 발표 (<https://github.com/newsroom/press-releases/agent-mode>): 여러 파일 변경, 조직별 지시사항, agentic coding workflow.
- Claude Code docs (<https://code.claude.com/docs>): codebase reading, command execution, MCP, instructions, hooks, subagents.
- Google DORA State of AI-assisted Software Development 2025 (<https://cloud.google.com/resources/content/2025-dora-ai-assisted-software-development-report>): AI 도입을 tools 문제가 아니라 software delivery system 문제로 다루는 관점.
- OpenAI Harness Engineering (<https://openai.com/index/harness-engineering/>): agent-first 개발에서 scaffolding, feedback loop, control system, evaluation harness가 중요해지는 흐름.
- SWE-bench Verified (<https://openai.com/index/introducing-swe-bench-verified/>): real-world GitHub issue 기반 coding agent 평가와 containerized harness.
- Why SWE-bench Verified no longer measures frontier coding capabilities (<https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>): 평가 데이터 오염과 테스트 설계 품질이 coding agent 평가에 미치는 영향.
- SWE-Lancer (<https://openai.com/index/swe-lancer/>): 실무 freelance software engineering task와 end-to-end test 기반 평가.
- Anthropic Effective harnesses for long-running agents (<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>): 긴 작업을 수행하는 agent에 필요한 초기화, 증분 진행, 다음 세션을 위한 산출물 설계.
- Anthropic Demystifying evals for AI agents (<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>): agent eval, regression suite, deterministic tests, static analysis, rubric 기반 평가 관점.
- Claude Code Best Practices (<https://code.claude.com/docs/en/best-practices>): verification, hooks, skills, subagents, permission/sandbox 설정을 활용하는 agentic coding 운영 방식.